Phase 8 Testing Requirements

Date: October 4, 2025
Repository: Mecca-Research/The-Binary-Decomposition-Interface
Purpose: Comprehensive testing requirements for Phase 8

Overview

This document defines the testing requirements for Phase 8, including test coverage targets, test scenarios, performance benchmarks, and fuzzing targets. The goal is to achieve **90%+ code coverage** with comprehensive test suites across all levels (unit, integration, system).

---

1. Test Coverage Requirements

1.1 Overall Coverage Target

Target: 90%+ code coverage across all components

Measurement: Using gcov/lcov

Breakdown by Component:

Component	Current Coverage	Target Coverage	Gap
VM Core	~60%	95%	+35%
JIT Compiler	~20%	90%	+70%
Garbage Collector	~70%	95%	+25%
Compiler (Lexer/Parser)	~50%	90%	+40%
Compiler (Codegen)	~30%	90%	+60%
Graph Execution	~10%	90%	+80%
Graph Optimization	~5%	85%	+80%
BCI/BTL	~80%	90%	+10%
Kernel Components	~40%	85%	+45%
Overall	**~45%**	**90%**	**+45%**

1.2 Coverage Exclusions

The following code is excluded from coverage requirements:
- Test code itself
- Debug-only code (`#ifdef DEBUG`)
- Unreachable error paths (assertions that should never trigger)
- Platform-specific code not relevant to current platform

1.3 Coverage Reporting

Tools:
- gcov for coverage data collection
- lcov for coverage report generation
- genhtml for HTML report generation

CI/CD Integration:
- Coverage reports generated on every PR
- Coverage regression detection (fail if coverage decreases)
- Coverage badge in README

---

2. Unit Test Requirements

2.1 VM Core Unit Tests

Target: 95% coverage, 50+ tests

Test Categories

A. Stack Operations (10 tests)
- Test push/pop operations
- Test stack overflow detection
- Test stack underflow detection
- Test stack reset
- Test stack inspection

B. Opcode Execution (30 tests)
- Test each opcode individually
- Test opcode with valid operands
- Test opcode with invalid operands
- Test opcode error handling
- Test opcode edge cases

C. Call Frame Management (10 tests)
- Test call frame creation
- Test call frame destruction
- Test call frame stack
- Test local variable access
- Test upvalue access

D. Global Variables (5 tests)
- Test global variable creation
- Test global variable access

- Test global variable update
- Test global variable deletion

E. Error Handling (5 tests)
- Test runtime error detection
- Test error message generation
- Test error recovery
- Test stack trace generation

2.2 GC Unit Tests

Target: 95% coverage, 40+ tests

Test Categories

A. Allocation (10 tests)
- Test object allocation
- Test string allocation
- Test array allocation
- Test allocation failure
- Test allocation statistics

B. Mark Phase (10 tests)
- Test root set scanning
- Test reachability marking
- Test cycle detection
- Test mark stack overflow
- Test mark statistics

C. Sweep Phase (10 tests)
- Test garbage collection
- Test free list rebuilding
- Test memory reclamation
- Test sweep statistics

D. Generational GC (10 tests)
- Test young generation collection
- Test old generation collection
- Test promotion logic
- Test remembered set
- Test write barriers

2.3 JIT Compiler Unit Tests

Target: 90% coverage, 40+ tests

Test Categories

A. IR Generation (20 tests)
- Test IR generation for each opcode
- Test IR generation for control flow
- Test IR generation for function calls
- Test IR generation for closures

B. Optimization Passes (10 tests)
- Test constant folding

- Test dead code elimination
- Test common subexpression elimination
- Test inlining

C. Hot Path Detection (5 tests)
- Test execution counting
- Test hot path threshold
- Test compilation triggering

D. Code Cache (5 tests)
- Test code caching
- Test code invalidation
- Test code eviction

2.4 Compiler Unit Tests

Target: 90% coverage, 50+ tests

Test Categories

A. Lexer (15 tests)
- Test token recognition
- Test keyword recognition
- Test operator recognition
- Test literal recognition
- Test error handling

B. Parser (20 tests)
- Test expression parsing
- Test statement parsing
- Test declaration parsing
- Test error recovery
- Test precedence handling

C. Semantic Analyzer (10 tests)
- Test type checking
- Test symbol resolution
- Test scope management
- Test error detection

D. Code Generator (15 tests)
- Test bytecode generation for each AST node
- Test jump patching
- Test constant emission
- Test local variable allocation

2.5 Graph Unit Tests

Target: 90% coverage, 40+ tests

Test Categories

A. Graph Construction (15 tests)
- Test node creation
- Test edge creation

- Test graph validation
- Test graph serialization

B. Graph Optimization (15 tests)
- Test constant folding
- Test dead node elimination
- Test common subexpression elimination
- Test subgraph fusion

C. Graph Execution (10 tests)
- Test topological sort
- Test graph-to-bytecode lowering
- Test graph execution
- Test device dispatch

---

3. Integration Test Requirements

3.1 VM Integration Tests

Target: 30+ tests

Test Scenarios

A. VM + GC Integration (10 tests)
1. Allocate objects through GC
2. Trigger GC collection
3. Verify garbage collection
4. Test root set scanning
5. Test write barriers
6. Test promotion
7. Test remembered set
8. Test allocation failure handling
9. Test GC statistics
10. Test GC performance

B. VM + JIT Integration (10 tests)
1. Detect hot path
2. Trigger JIT compilation
3. Execute compiled code
4. Test interpreter→native transition
5. Test native→interpreter transition
6. Test deoptimization
7. Test code caching
8. Test compilation statistics
9. Test JIT performance
10. Test JIT correctness

C. VM + Bytecode Integration (10 tests)
1. Load bytecode chunk
2. Execute bytecode
3. Test control flow

4. Test function calls
5. Test variable access
6. Test error handling
7. Test stack management
8. Test call frame management
9. Test global variables
10. Test closures

3.2 Compiler Integration Tests

Target: 20+ tests

Test Scenarios

A. Lexer + Parser (5 tests)
1. Lex and parse simple expression
2. Lex and parse complex expression
3. Lex and parse statement
4. Lex and parse function
5. Lex and parse program

B. Parser + Semantic Analyzer (5 tests)
1. Parse and analyze types
2. Parse and analyze symbols
3. Parse and analyze scopes
4. Parse and analyze errors
5. Parse and analyze control flow

C. Semantic Analyzer + Codegen (5 tests)
1. Analyze and generate expression
2. Analyze and generate statement
3. Analyze and generate function
4. Analyze and generate program
5. Analyze and generate errors

D. Compiler + VM (5 tests)
1. Compile and execute expression
2. Compile and execute statement
3. Compile and execute function
4. Compile and execute program
5. Compile and execute errors

3.3 Graph Integration Tests

Target: 20+ tests

Test Scenarios

A. Graph + Optimization (10 tests)
1. Build graph and optimize
2. Test constant folding
3. Test dead node elimination
4. Test common subexpression elimination
5. Test subgraph fusion
6. Test optimization correctness

7. Test optimization performance
8. Test optimization statistics
9. Test optimization errors
10. Test optimization edge cases

B. Graph + VM (10 tests)
1. Build graph and execute
2. Test graph-to-bytecode lowering
3. Test graph execution correctness
4. Test graph execution performance
5. Test graph execution errors
6. Test graph execution edge cases
7. Test device dispatch
8. Test data flow
9. Test control flow
10. Test memory management

---

4. End-to-End Test Requirements

4.1 Complete Workflow Tests

Target: 30+ tests

Test Scenarios

A. Source → Bytecode → Execution (10 tests)
1. Hello World program
2. Fibonacci (recursive)
3. Factorial (iterative)
4. Array sum
5. Nested functions
6. Control flow (if/while/for)
7. Closures
8. Recursion
9. Error handling
10. Complex program

B. Source → JIT → Native Execution (10 tests)
1. Simple function (hot path)
2. Loop (hot path)
3. Recursive function (hot path)
4. Matrix multiply (hot path)
5. String processing (hot path)
6. Array operations (hot path)
7. Nested loops (hot path)
8. Function calls (hot path)
9. Closures (hot path)
10. Complex program (hot path)

C. Graph → Optimization → Execution (10 tests)
1. Simple graph

2. Linear graph
3. DAG graph
4. Cyclic graph (error)
5. Large graph
6. Optimized graph
7. Fused graph
8. Device-dispatched graph
9. Multi-device graph
10. Complex graph

4.2 Error Handling Tests

Target: 20+ tests

Test Scenarios

A. Compilation Errors (10 tests)
1. Syntax error
2. Type error
3. Undefined variable
4. Undefined function
5. Duplicate declaration
6. Invalid operation
7. Invalid type
8. Invalid scope
9. Invalid control flow
10. Invalid expression

B. Runtime Errors (10 tests)
1. Division by zero
2. Stack overflow
3. Stack underflow
4. Out of memory
5. Null pointer dereference
6. Array out of bounds
7. Type mismatch
8. Invalid function call
9. Invalid operation
10. Assertion failure

---

5. Stress Test Requirements

5.1 Stress Test Scenarios

Target: 10+ tests

Test Categories

A. Large Program Compilation (2 tests)
1. Compile 10,000+ line program
2. Compile 100,000+ line program

B. Deep Recursion (2 tests)
1. Recursive function with 1,000+ levels
2. Recursive function with 10,000+ levels

C. Large Graph Execution (2 tests)
1. Execute graph with 10,000+ nodes
2. Execute graph with 100,000+ nodes

D. Memory Pressure (2 tests)
1. Allocate 1GB+ objects
2. Allocate 10GB+ objects

E. Long-Running Execution (2 tests)
1. Execute program for 1+ hour
2. Execute program for 24+ hours

5.2 Stress Test Acceptance Criteria

- No crashes
- No memory leaks
- No performance degradation over time
- Correct results
- Reasonable resource usage

---

6. Performance Benchmark Requirements

6.1 Benchmark Scenarios

Target: 15+ benchmarks

Benchmark Categories

A. Computation Benchmarks (5 benchmarks)
1. Fibonacci (recursive)
2. Fibonacci (iterative)
3. Matrix multiplication
4. Sorting (quicksort, mergesort)
5. Graph traversal (BFS, DFS)

B. Memory Benchmarks (5 benchmarks)
1. Object allocation
2. Object deallocation
3. GC collection
4. Memory access patterns
5. Cache performance

C. Compilation Benchmarks (5 benchmarks)
1. Lexing speed
2. Parsing speed
3. Semantic analysis speed
4. Code generation speed
5. JIT compilation speed

6.2 Performance Targets

Benchmark	Target	Baseline
Fibonacci (recursive, n=30)	< 1s	TBD
Fibonacci (iterative, n=1000000)	< 0.1s	TBD
Matrix multiply (1000x1000)	< 5s	TBD
Quicksort (1000000 elements)	< 2s	TBD
Object allocation (1000000 objects)	< 1s	TBD
GC collection (1000000 objects)	< 0.5s	TBD
Compile 1000 line program	< 0.1s	TBD
JIT compile function	< 0.01s	TBD

6.3 Performance Comparison

Interpreter vs JIT:
- Target: 2-5x speedup for hot loops
- Target: 10-50x speedup for numeric computation

GC Performance:
- Target: < 10ms pause time for young generation
- Target: < 100ms pause time for full collection

---

7. Fuzzing Requirements

7.1 Fuzzing Targets

Target: 5+ fuzzing harnesses

Fuzzing Categories

A. Lexer Fuzzing
- Input: Random byte sequences
- Target: Lexer should not crash
- Duration: 24+ hours

B. Parser Fuzzing
- Input: Random token sequences
- Target: Parser should not crash
- Duration: 24+ hours

C. VM Fuzzing
- Input: Random bytecode sequences
- Target: VM should not crash
- Duration: 24+ hours

D. JIT Fuzzing
- Input: Random bytecode sequences
- Target: JIT should not crash
- Duration: 24+ hours

E. Graph Fuzzing
- Input: Random graph structures
- Target: Graph execution should not crash
- Duration: 24+ hours

7.2 Fuzzing Acceptance Criteria

- 0 crashes after 24+ hours
- 0 hangs after 24+ hours
- 0 memory leaks after 24+ hours
- All discovered bugs fixed

7.3 Fuzzing Infrastructure

Tools:
- AFL (American Fuzzy Lop)
- LibFuzzer
- Honggfuzz

Corpus:
- Initial corpus: Valid inputs
- Corpus minimization: Remove redundant inputs
- Corpus growth: Add interesting inputs

Continuous Fuzzing:
- Run fuzzing on CI/CD
- Report crashes automatically
- Track fuzzing coverage

---

8. Regression Test Requirements

8.1 Regression Test Suite

Target: All existing tests + new tests

Test Categories

A. Unit Tests
- All unit tests from Phase 8.2

B. Integration Tests
- All integration tests from Phase 8.2

C. End-to-End Tests
- All end-to-end tests from Phase 8.2

D. Performance Tests
- All performance benchmarks from Phase 8.2

8.2 Regression Detection

Functional Regression:
- Any test failure is a regression
- Automated detection on every PR

Performance Regression:
- > 10% slowdown is a regression
- Automated detection on every PR

Coverage Regression:
- Any coverage decrease is a regression
- Automated detection on every PR

8.3 Regression CI/CD

On Every PR:
1. Run all unit tests
2. Run all integration tests
3. Run all end-to-end tests
4. Run performance benchmarks
5. Generate coverage report
6. Detect regressions
7. Report results

On Every Merge:
1. Run full regression suite
2. Run extended performance benchmarks
3. Run fuzzing (short duration)
4. Update baseline metrics

---

9. Test Infrastructure Requirements

9.1 Test Framework

Requirements:
- Simple test definition
- Test discovery
- Test execution
- Test reporting
- Test fixtures (setup/teardown)
- Test assertions
- Test mocking (if needed)

Recommended: Custom lightweight framework or existing C test framework (e.g., Unity, CMocka)

9.2 Test Utilities

Required Utilities:
- Memory leak detection (valgrind)
- Address sanitizer (ASan)
- Undefined behavior sanitizer (UBSan)
- Memory sanitizer (MSan)
- Thread sanitizer (TSan)
- Code coverage (gcov/lcov)
- Performance profiling (perf, gprof)

9.3 Test Automation

CI/CD Integration:
- Automated test execution on every PR
- Automated test execution on every merge
- Automated nightly test runs
- Automated weekly fuzzing runs

Test Reporting:
- Test results dashboard
- Coverage reports
- Performance reports
- Fuzzing reports

---

10. Test Documentation Requirements

10.1 Test Documentation

Required Documentation:
- Testing guide (how to write tests)
- Testing best practices
- Test coverage requirements
- Test execution instructions
- Test debugging guide

10.2 Test Code Quality

Requirements:
- Tests should be readable
- Tests should be maintainable
- Tests should be fast (< 1s per test)
- Tests should be isolated (no dependencies)
- Tests should be deterministic (no flakiness)

---

11. Success Criteria

Phase 8.2 Success Criteria

- ✅ 90%+ code coverage achieved

- ✅ 150+ unit tests passing
- ✅ 50+ integration tests passing
- ✅ 30+ end-to-end tests passing
- ✅ 10+ stress tests passing
- ✅ 15+ performance benchmarks established
- ✅ 5+ fuzzing harnesses operational
- ✅ 120+ hours of fuzzing completed
- ✅ 0 known crashes
- ✅ 0 known memory leaks
- ✅ Regression suite automated
- ✅ Test infrastructure complete

---

12. Timeline

Week 6: Test Infrastructure (PR #109)

- Set up code coverage
- Add 100+ unit tests
- Achieve 90%+ coverage

Week 7: Integration Tests (PR #110)

- Add 50+ integration tests
- Test all major workflows

Week 8: Fuzzing & Stress (PR #111, #112)

- Set up fuzzing infrastructure
- Run fuzzing campaigns
- Add stress tests
- Add regression suite

Conclusion

This document defines comprehensive testing requirements for Phase 8. By following these requirements, we will achieve 90%+ code coverage, comprehensive test suites, and a robust, reliable BDI system.

Next Steps: Begin implementing test infrastructure in PR #109.